

XStock: Warehouse Inventory Management System

Project Report Documentation

Developed using NestJS, Prisma, PostgreSQL, and Next.js

Author: Kresna Winata Perwiro Negoro Student ID: 5025231308

Department of Informatics Engineering Institut Teknologi Sepuluh Nopember (ITS)

Abstract

Warehouse operations in distribution companies often suffer from fragmented records, manual stock tracking, and poor visibility over product movement, especially when products are stored in multiple sections, tracked by batch, and constrained by expiry dates. These issues can lead to overstocking, stockouts, and operational errors during inbound and outbound processes.

XStock is a web-based Warehouse Inventory Management System designed to address these challenges. The system supports end-to-end workflows for both purchasing and sales activities through two main flows: Purchase Order → Inbound → Goods Receipt and Sales Order → Outbound → Shipment. XStock tracks stock at the level of product, batch, and storage location (section), and records every movement using auditable stock movement entries.

The system is implemented using a modern full-stack architecture. The backend is built with NestJS, Prisma, and PostgreSQL, while the frontend uses Next.js with the App Router. Role-based access control and JWT-based authentication are used to separate administrative tasks from operational tasks. The system also supports local file uploads (e.g., product images or documents) and provides a dashboard with key warehouse statistics and recent activities.

This report describes the requirements, architecture, database design, workflows, implementation details, testing strategy, and deployment considerations for XStock. The documentation aims to provide a clear technical reference for future maintenance, extension, and evaluation of the system.

Contents

Abstract	2
1 Introduction	5
1.1 Background	5
1.2 Problem Statement	5
1.3 Objectives	5
1.4 Scope and Limitations	6
1.5 Methodology Overview	6
2 System Requirements	7
2.1 Functional Requirements	7
2.2 Non-Functional Requirements	7
2.3 User Roles	8
3 System Design and Architecture	9
3.1 Technology Stack	9
3.2 High-Level Architecture	9
3.3 Module Overview (Backend)	9
4 Business Processes and Workflows	10
4.1 Purchase Order to Goods Receipt Workflow	10
4.2 Sales Order to Shipment Workflow	10
4.3 Inventory and Stock Movement	11
4.4 Batch and Expiry Management	11
5 Database Design	12
5.1 Conceptual Overview	12
5.2 Example Prisma Models (Simplified)	12
5.3 Referential Integrity and Constraints	13
6 Backend Implementation (NestJS + Prisma)	14
6.1 Backend Project Structure	14
6.3 Core Backend Business Logic	14
Goods Receipt Logic	14
Shipment Logic	14
Inventory Consistency Rules	14
7 Frontend Implementation (Next.js)	16
7.1 Routing Structure	16
7.2 API Helper	16
7.3 Key Screens	16
Dashboard	16
Products	16
Inventory	16
Purchase Orders & Goods Receipts	17
Sales Orders & Shipments	17
Reports	17
7.4 UX Considerations	17
8 Security, Authentication, and Authorization	18
8.1 JWT Authentication	18
8.2 Role-Based Access Control	18
8.3 Data Protection Practices	18
9 Testing and Quality Assurance	19
9.1 Unit & Integration Testing	19

9.2 End-to-End (E2E) Workflow Testing	19
9.3 Manual Testing	19

1 Introduction

1.1 Background

Distribution companies rely on accurate and timely information about stock levels in their warehouses. Products can be stored in multiple sections, associated with different batches, and limited by expiry dates. When these aspects are not tracked centrally and consistently, serious problems may appear:

- Overstocking and unnecessary holding costs
- Stockouts and inability to fulfill customer orders
- Expired products remaining in inventory
- Difficulties in tracing product movement for auditing

Traditional spreadsheet-based or manual approaches are prone to human error and do not scale well as the number of products, suppliers, and customers grows. A dedicated warehouse management system (WMS) is required to ensure operational efficiency and traceability.

XStock is developed to provide a structured warehouse management solution focusing on batch tracking, expiry monitoring, and full traceability of stock movements aligned with common distribution workflows.

1.2 Problem Statement

The main problems that XStock aims to solve are:

- Lack of a central system to manage products, batches, locations, and stock levels.
- No integrated workflow connecting Purchase Orders to Goods Receipts, and Sales Orders to Shipments.
- Weak traceability of stock movement, making it difficult to answer questions such as “Why is this stock level like this?” or “Which shipment consumed this batch?”.
- Limited visibility for different user roles (e.g., admin vs. staff) regarding what actions they are allowed to perform.

1.3 Objectives

The objectives of XStock are:

1. To design and implement a web-based warehouse management system for distribution companies.
2. To support complete inbound and outbound workflows:
 - Purchase Order → Goods Receipt.
 - Sales Order → Shipment.
3. To track stock at the level of product, batch, and location.
4. To maintain a detailed log of stock movements (IN, OUT, TRANSFER).
5. To implement role-based authentication and authorization for secure access.
6. To provide a dashboard for monitoring warehouse performance and recent activities.

1.4 Scope and Limitations

Scope of the system:

- Product master data management (SKU, name, unit of measure, etc.).
- Storage location management (sections or zones in the warehouse).
- Purchase Order and Goods Receipt handling.
- Sales Order and Shipment handling.
- Stock inventory overview per product, batch, and location.
- Batch code and expiry date tracking.
- File uploads for product images or supporting documents.

Limitations:

- The system is focused on a single warehouse context. Multi-warehouse support is not fully implemented.
- Advanced features such as automated replenishment planning, integration with external ERP systems, or barcode/QR scanning are outside the current scope but can be added in future work.

1.5 Methodology Overview

The development of XStock follows a simplified software engineering life cycle:

1. Requirement analysis through scenario-based thinking for a distribution warehouse.
2. System design including architecture, database schema, and workflow definitions.
3. Implementation using NestJS, Prisma, PostgreSQL, and Next.js.
4. Testing, including unit tests, integration tests, and selected end-to-end tests.
5. Deployment in a development or production environment using environment variables and process managers.

2 System Requirements

2.1 Functional Requirements

Key functional requirements of XStock include:

- User authentication and authorization:
 - Users can log in using an email and password.
 - Different roles (e.g., Admin, Staff) have different permissions.
- Product management:
 - Admin can create, update, and deactivate products.
 - Products include data such as SKU, name, unit of measure, and optional image.
- Location management:
 - Admin can create and manage storage locations or sections.
 - Inventory is tracked at the (product, batch, location) combination.
- Purchase Order management:
 - Admin can create Purchase Orders with multiple items.
 - Purchase Orders track ordered quantities and supplier information.
- Goods Receipt management:
 - Users can create Goods Receipts referencing Purchase Orders.
 - Quantities received must not exceed the ordered quantities.
 - Goods Receipts create INBOUND stock movements.
- Sales Order management:
 - Admin or Staff can create Sales Orders for customers.
 - Sales Orders contain one or more items with requested quantities.
- Shipment management:
 - Users can create Shipments referencing Sales Orders.
 - Shipments reduce stock through OUTBOUND movements.
 - Partial shipments are allowed; shipped quantities are accumulated.
- Inventory view:
 - Users can see available stock per product, batch, and location.
 - Users can filter and search by product, batch, or section.
- Stock movement log:
 - Each change in stock is recorded with a movement type (INBOUND, OUTBOUND, TRANSFER).
 - Movements reference their source (Goods Receipt or Shipment).

2.2 Non-Functional Requirements

Important non-functional requirements include:

- **Performance:** API responses for normal CRUD operations should be fast enough for interactive use in a browser.
- **Reliability:** Referential integrity is enforced by the database using foreign keys. Critical operations use transactions.

- **Security:** Passwords are hashed; JWT is used for authentication; role-based guard is used for authorization.
- **Maintainability:** Code is organized into feature modules in NestJS and separate pages/components in Next.js.
- **Scalability:** The system can be extended with additional modules or deployed on scalable infrastructure.

2.3 User Roles

The system distinguishes at least two main roles:

- **Admin**
 - Manage master data: products, locations.
 - Manage users and roles (optional extension).
 - Create and approve Purchase Orders and Sales Orders.
 - View all reports and dashboards.
- **Staff**
 - Create and process Goods Receipts based on Purchase Orders.
 - Create and process Shipments based on Sales Orders.
 - View inventory and perform transfers if allowed.

Future extensions may introduce additional roles such as “Manager” or “Auditor”, but the current implementation focuses on Admin and Staff.

3 System Design and Architecture

3.1 Technology Stack

- **Backend:** NestJS (TypeScript), using:
 - Controllers, Services, and Modules for clean separation.
 - Prisma as the ORM.
 - PostgreSQL as the relational database.
- **Frontend:** Next.js (App Router), using:
 - React components.
 - Client-side pages for interactive dashboards and forms.
 - A custom api helper to communicate with the backend.
- **Database:** PostgreSQL, chosen for its reliability and strong support for relational constraints.
- **Authentication:** JWT-based authentication with role-based authorization using NestJS guards.
- **File Storage:** Local directory (e.g., uploads/) exposed via static path from NestJS.

3.2 High-Level Architecture

The following description conceptually shows the high-level architecture of XStock:

- Frontend (Next.js) ↔ Backend (NestJS) ↔ PostgreSQL
- Backend ↔ Local Uploads directory

In more detail:

- The frontend runs as a Next.js application, sending HTTP requests to the NestJS backend using JSON.
- The backend exposes RESTful endpoints for authentication, products, orders, shipments, inventory, and reports.
- Prisma acts as the data access layer, translating TypeScript code into SQL queries.
- Static files such as uploaded images are served from a dedicated folder.

3.3 Module Overview (Backend)

The backend is split into multiple logical modules, for example:

- AuthModule – handles login, JWT creation, and guards.
- UserModule – manages user entities and roles (if included).
- ProductModule – CRUD operations for products.
- LocationModule – storage location management.
- PurchaseModule – Purchase Orders and Goods Receipts.
- OutboundModule – Sales Orders and Shipments.
- InventoryModule – aggregation and queries over inventory and stock movement.
- UploadModule – configuration of file uploads and static asset serving.
- DashboardModule – high-level metrics and recent activity logs.

Each module follows NestJS conventions, having its own controller and service.

4 Business Processes and Workflows

4.1 Purchase Order to Goods Receipt Workflow

The inbound process starts with creating a Purchase Order (PO) for a supplier and ends with a Goods Receipt (GR) and updated stock.

Step-by-step:

1. Create Purchase Order

- Admin records the supplier, order date, and the list of products with quantities.
- The PO is stored with status such as OPEN.

2. Supplier delivers goods

- Physical products arrive at the warehouse.

3. Create Goods Receipt

- Staff checks the delivered goods and creates a Goods Receipt referencing the PO.
- For each product, the received quantity is entered, along with batch information and expiry dates if relevant.

4. Validation and Stock Update

- The system verifies that the cumulative received quantity for each item does not exceed the ordered quantity.
- If valid, the system:
 - Creates INBOUND StockMovement records.
 - Updates Inventory entries by increasing quantities in the corresponding (product, batch, location).

5. Update Purchase Order Status

- If all items are fully received, the PO is marked as COMPLETED or CLOSED.
- If partially received, it may remain PARTIALLY_RECEIVED.

This workflow ensures that purchasing and inventory updates are tightly connected and that no goods are added without a corresponding Purchase Order.

4.2 Sales Order to Shipment Workflow

The outbound process starts with a Sales Order (SO) from a customer and ends with a Shipment and reduced stock.

Step-by-step:

1. Create Sales Order

- Admin or Staff records the customer, order date, and the desired products with quantities.
- The SO is stored, initially with status like OPEN.

2. Check Availability

- Before creating a shipment, staff can inspect inventory to ensure that the requested items are available.

3. Create Shipment

- A Shipment is created referencing the Sales Order.
- For each line, staff selects the quantity to ship, possibly using specific batches and locations.

4. Validation and Stock Deduction

- The system verifies that the total shipped quantity for each Sales Order item does not exceed the ordered quantity.
- The system also ensures that there is enough stock in the chosen locations and batches.
- If valid, the system:
 - Creates OUTBOUND StockMovement records.
 - Decreases the Inventory quantities accordingly.

5. Update Sales Order Status

- If the entire order has been shipped, the SO status becomes COMPLETED or DELIVERED.
- If some items remain unshipped, the status may be PARTIALLY_SHIPPED.

This workflow ensures that stock is never shipped beyond what has been ordered and maintains an audit trail of each shipment.

4.3 Inventory and Stock Movement

Inventory in XStock is not just a single quantity per product. Instead, it is recorded as:

- Product
- Optional Batch
- Location (section)
- Quantity

Whenever stock changes, a StockMovement entry is created with fields such as:

- Movement type: INBOUND, OUTBOUND, TRANSFER.
- Reference type: e.g., GOODS_RECEIPT, SHIPMENT.
- Reference ID: ID of the associated Goods Receipt or Shipment.
- Product, batch, location, and quantity.
- Timestamp and user who performed the operation.

This design enables detailed auditability and traceability over time.

4.4 Batch and Expiry Management

For products that need batch tracking and expiry dates (e.g., perishable goods or regulated items):

- A ProductBatch entity is used to represent each batch, with fields such as:
 - batchCode
 - expiryDate
 - productId
- Inventory references these batches to indicate exactly which batch is stored at which location.
- Reports can identify:
 - Near-expiry stock.
 - Expired stock.
 - Batches used in specific shipments.

5 Database Design

5.1 Conceptual Overview

At a high level, the database is centered around the following main concepts:

- Product
- Location (section)
- ProductBatch
- Inventory
- PurchaseOrder and PurchaseOrderItem
- GoodsReceipt and GoodsReceiptLine
- SalesOrder and SalesOrderItem
- Shipment and ShipmentLine
- StockMovement
- User (for authentication and roles)

These entities are connected through foreign keys to maintain referential integrity.

5.2 Example Prisma Models (Simplified)

Below is a simplified representation of some important Prisma models used in the system:

```
// Product master data
model Product {
  id          String          @id @default(uuid())
  name        String
  sku         String          @unique
  uom         String
  batches     ProductBatch[]
  createdAt   DateTime        @default(now())
  updatedAt   DateTime        @updatedAt
}

// Batch information
model ProductBatch {
  id          String          @id @default(uuid())
  batchCode   String
  expiryDate  DateTime?
  product     Product         @relation(fields: [productId], references: [id])
  productId   String
  inventory   Inventory[]
}

// Location / Section information
model Location {
  id          String          @id @default(uuid())
  code        String
  type        String          // e.g. "SECTION"
  inventory   Inventory[]
}

// Inventory state at (product, batch, location)
```

```

model Inventory {
  id          String      @id @default(uuid())
  productId   String
  batchId     String?
  locationId  String
  quantity    Int         @default(0)
  product     Product     @relation(fields: [productId], references: [id])
  batch       ProductBatch @relation(fields: [batchId], references: [id])
  location    Location     @relation(fields: [locationId], references: [id])
}

```

The other models (PurchaseOrder, GoodsReceipt, SalesOrder, Shipment, StockMovement) follow similar relational patterns with clear parent–child relationships.

5.3 Referential Integrity and Constraints

- Foreign key constraints ensure that:
 - A GoodsReceiptLine cannot exist without a referenced GoodsReceipt.
 - A ShipmentLine cannot exist without a referenced Shipment.
 - A StockMovement must reference valid product, batch, and location records.
- Deletion rules are carefully designed:
 - High-level entities such as Purchase Orders and Goods Receipts should not be deleted once dependent records exist, preserving audit trails.
 - Status fields (e.g., CANCELLED) are preferred over physical deletion.
 - Inventory changes always occur through validated movement transactions (NEVER direct edits).

6 Backend Implementation (NestJS + Prisma)

6.1 Backend Project Structure

A representative backend structure is as follows:

```
src/  
├── main.ts  
├── app.module.ts  
├── prisma.service.ts  
├── auth/  
├── users/  
├── products/  
├── locations/  
├── purchase/  
├── outbound/  
├── inventory/  
├── dashboard/  
└── upload/
```

6.3 Core Backend Business Logic

The backend enforces strict consistency rules to ensure data integrity throughout all warehouse workflows.

Goods Receipt Logic

- A Goods Receipt cannot exceed the remaining ordered quantity.
- Stock increments ONLY occur through validated Goods Receipt creation.
- When a GR is created:
 - INBOUND StockMovement entries are generated.
 - Inventory quantities increase for (product, batch, location).
 - Purchase Order item receivedQty values update automatically.
 - Purchase Order status transitions:
 - OPEN → PARTIAL → COMPLETED.

Shipment Logic

- A Shipment cannot exceed ordered quantity.
- A Shipment cannot exceed on-hand inventory.
- When a Shipment is created:
 - OUTBOUND StockMovement entries are generated.
 - Inventory quantities decrease accordingly.
 - Sales Order item shippedQty values update automatically.
 - Sales Order status transitions:
 - OPEN → PARTIAL → DELIVERED.

Inventory Consistency Rules

- Inventory is tracked per:

- product
- batch
- location
- Direct modification of Inventory is NOT allowed.
- All updates must originate from:
 - Goods Receipts
 - Shipments
 - Transfers (if implemented)

7 Frontend Implementation (Next.js)

7.1 Routing Structure

```
/app/  
  dashboard/  
  products/  
  inventory/  
  purchase-orders/  
  goods-receipts/  
  sales-orders/  
  shipments/  
  reports/
```

Each route folder contains a `page.tsx` entrypoint.

7.2 API Helper

The frontend centralizes backend communication through a custom API wrapper. This helper injects:

- the base backend URL
- JSON headers
- authorization token
- unified error handling

Example conceptual behavior: `api(path, method: "GET" | "POST", body)`

This ensures all network communication follows a consistent pattern throughout the application.

7.3 Key Screens

The primary user-interface screens include:

Dashboard

- Summary metrics
- Recent Purchase Orders, Goods Receipts, Shipments
- Critical inventory alerts (optional)

Products

- Product listing with search and pagination
- CRUD operations
- Support for product images stored in `/uploads/`

Inventory

- Real-time grouped stock view by product, batch, and location
- Filters for SKU, product name, section
- Optional expiration tracking

Purchase Orders & Goods Receipts

- PO creation and status monitoring
- GR creation with validation against remaining ordered quantity
- Automatic update of received quantities and PO status

Sales Orders & Shipments

- SO creation and progress tracking
- Shipment creation enforcing inventory + order constraints
- Automatic update of shipped quantities and SO status

Reports

- Warehouse performance metrics
- Shipment and receipt volume charts
- Expiry and batch analytics (optional extensions)

7.4 UX Considerations

The interface prioritizes:

- Clear typography and spacing
- Intuitive navigational hierarchy
- Real-time validation feedback
- Minimal user friction during operational tasks

8 Security, Authentication, and Authorization

8.1 JWT Authentication

Authentication uses JSON Web Tokens (JWT):

- User logs in → receives JWT
- JWT persists client-side (memory or storage)
- Each protected request includes the token in the Authorization header
- Backend guards validate token integrity and expiration

8.2 Role-Based Access Control

Two main roles are implemented:

- Admin
 - Full access to master data (products, locations)
 - Can create & manage POs and SOs
 - Full reporting access
- Staff
 - Limited to operational tasks: GR and Shipment creation
 - View-only access for master data

Authorization is enforced using NestJS custom guards and metadata.

8.3 Data Protection Practices

Security principles applied:

- Password hashing using bcrypt
- Sensitive fields omitted from responses
- File uploads restricted by MIME type
- HTTPS recommended for production
- Optional rate limiting

9 Testing and Quality Assurance

9.1 Unit & Integration Testing

Unit testing verifies:

- Business logic edge cases
- Quantity validation (order vs received vs shipped)
- Status transitions for POs and SOs

Integration testing validates:

- Interaction between NestJS services and Prisma
- Correct persistence of Goods Receipts and Shipments
- Automatic stock movement generation

9.2 End-to-End (E2E) Workflow Testing

A comprehensive E2E flow includes:

- Login → obtain JWT
- Create Product and Location
- Create Purchase Order
- Create Goods Receipt
- Validate stock increase
- Create Sales Order
- Create Shipment
- Validate stock decrease and SO status update

This ensures each subsystem integrates correctly in real-world scenarios.

9.3 Manual Testing

Manual QA includes:

- Form validation feedback
- Prevention of invalid operations (over-ship, over-receive)
- Upload preview testing
- Navigation and UI responsiveness checks